Téma 7: Projekt — celý pipeline (00–10)

📖 Čo sa tu naučím

- Celý WUM pipeline od 12 GB Apache logu po doplnené sedenia (11 úloh)
- Vzorce: `RLength = -ln(1-p)*mean`, `STT_Q = Q3+1.5*IQR`, `Length = UNIXTIME_i - UNIXTIME_(i-1)`
- KS test (Kolmogorov-Smirnov) — kedy RLength, kedy STT_Q fallback
- Pridelené obdobie: 17.12.–23.12.2017 (Martin Studený), 7 dní
- Edge cases: NaN handling pre prvý záznam usera, browser cache → path completion

🔑 Najdôležitejšie vzorce a koncepty

```
Length_i      = UNIXTIME_i - UNIXTIME_(i-1)     (per USER_ID, NaN pre 1. záznam)
RLength       = -ln(1-p) * mean(Length)         (pre p=0.4 ≈ 0.5108 * mean)
STT_Q         = Q3 + 1.5 * IQR                  (Tukeyho horná ohrada)
STT (60 min)  = 3600 s                          (prah pre nové sedenie)
KS test       = H0: data ~ Exp(scale=mean); p < 0.05 → zamietame
USER_ID       = unikátna dvojica (IP, Agent)    (Cooley heuristika)
SESSION_ID    = cumsum(NEW_SESSION)             (vektorizovaný idiom)
```

Z prednášky 1 (Proces objavovania znalostí): *„Objavovanie znalostí chápeme ako proces, ktorý zahŕňa výber dát, predspracovanie dát, transformáciu dát, analýzu dát a interpretáciu výsledkov. Cieľom objavovania znalostí na základe používania webu (Web Log Mining) je analýza správania sa používateľov pri prechádzaní webu."* — náš celý pipeline 00–10 je presne fáza **prípravy dát** podľa metodiky CRISP-DM.

Z prednášky 6 a 8 (Fázy procesu — príprava dát): *„Proces sa rozdeľuje do troch základných častí: získanie súboru z webového servera → čistenie súboru od nepotrebných dát a robotov → identifikácia používateľov a sedení (na základe rôznej verzie prehliadača, štruktúry stránok, času) → doplnenie ciest."* — toto je presne poradie úloh 00 → 10 v našom projekte.

Z prednášky 8 (kvantilový odhad hraničného času): *„Ak premenná X nemá exponenciálne rozdelenie, na odhad časového okna τ môžeme použiť kvartilový odhad τ = Q = Q_3 + 1,5·IQR, kde Q_3 je horný kvartil (75. percentil) a IQR je kvartilové rozpätie (stredných 50 % hodnôt)."*

Z prednášky 7 a 8 (RLength — referenčná dĺžka): *„Premenná X má exponenciálne rozdelenie f(x) = λe^(−λx), F(x) = 1 − e^(−λx). Ak γ je relatívna početnosť navigačných stránok, na odhad hraničného času τ môžeme použiť kvantilovú funkciu $F^{-1}(\gamma) = \tau = -\ln(1-\gamma)/\lambda$. Maximálne vierohodný odhad parametra λ (priemerná intenzita udalostí) je λ = 1/x̄, kde x̄*

> *je pozorovaný priemer dĺžky návštev.*" — odtiaľ priamo plynie `RLength = -ln(1-p) * mean(Length)`.

Úloha 00: extract_week.py

Teoretický základ: Prvá fáza CRISP-DM je *získanie súboru z webového servera* a redukcia hrubých dát na pracovnú vzorku. Prednáška 3 ukazuje formát Apache logu (`178.41.1.187 - - [16/Dec/2019:18:16:26 +0100] "GET ..."`), z ktorého filtrujeme len pridelený týždeň. Cieľom je dostať dáta do veľkosti, ktorá sa zmestí do RAM pre ďalšie spracovanie pandasom. *(viď prednáška 3 — Záznam logovacieho súboru; prednáška 6 — Fázy procesu, získanie súboru z webového servera)*

Čo robí: Vystrihne zo 12 GB master logu len pridelený týždeň (17.–23.12.2017) → ~216 MB súbor.

Vstup → výstup: `../../materials/wm2020projekt.log` (12 GB, ~47M riadkov) → `../01_cistenie_zaznamov/in/wm2020projekt_studeny.log` + kópia do `02_robot_ips/in/`.

Kľúčové kroky: 1. Definuj 7 substringov dní `"17/Dec/2017"`..`"23/Dec/2017"` ako tuple 2. Stream čítaj 12 GB log (`for line in fin`), O(1) pamäť 3. Substring test `if d in line: write` + `break` (max 1 deň na riadku) 4. `shutil.copy()` výsledok do druhého `in/` (bytewise, OS-level rýchle)

Dôležité parametre: - `encoding="utf-8", errors="replace"` — neplatné bajty → U+FFFD (nepadá) - `tuple` namiesto `list` — immutability + mikro-rýchlosť pri 47M iteráciach - `break` po prvom matchi — ušetrí ~3.5 zbytočných porovnaní

Vzorec / formula: žiadny — len substring match.

Časté otázky pri obhajobe: - **Q:** Prečo nepoužívaš pandas/regex na dátum? **A:** 12 GB sa nezmestí do RAM; `'in'` operátor je C-úroveň memmem, rýchlejší ako regex. - **Q:** Koľko záznamov ostane? **A:** ~909 828 (audit overené). - **Q:** Kde v CRISP-DM patrí tento krok? **A:** Do fázy *získanie súboru z webového servera* — prvá z troch základných častí Web Log Miningu *(viď prednáška 6)*.

Úloha 01: parsovanie_a_cistenie.py

Teoretický základ: Druhá fáza CRISP-DM je *čistenie súboru od nepotrebných dát*. Prednáška 3 popisuje Apache Combined Log Format: `IP - - [DateTime +TZ] "METHOD URL HTTP/X" status bytes "Referrer" "User-Agent"`. Z neho potrebujeme štruktúrovaný DataFrame so 7 stĺpcami a odstrániť všetko, čo nereprezentuje aktívnu navigáciu používateľa (statické súbory, neúspešné

requesty, iné metódy ako GET). *(viď prednáška 3 — Záznam logovacieho súboru; prednáška 6 — Čistenie súboru od nepotrebných dát)*

Čo robí: Z Apache Combined Log Format (raw text) urobí štruktúrovaný CSV so 7 stĺpcami + 3 filtre.

Vstup → výstup: `in/wm2020projekt_studeny.log` (~910k riadkov) → `out/01_cleaned_log.csv` (~211 313 riadkov).

Kľúčové kroky: 1. Skompiluj regex (8 capture groups) pre Apache CLF 2. Skompiluj regex statických prípon (jpg/css/js/png/woff/...) 3. Stream čítaj log + `pattern.match(line)` 4. Filtre: `method=="GET"`, `200<=status<=399`, URL nie statická prípona 5. Zber do `list[dict]` → `pd.DataFrame` → `to_csv`

Dôležité parametre: - `re.IGNORECASE` — `.JPG` aj `.jpg` matchne - `(?:\?|$)` — prípona musí byť na konci URL alebo pred `?query` - `list[dict]` → DataFrame O(N) (vs. `df.append()` $O(N^2)$)

Vzorec / formula: žiadny — len filtre.

Časté otázky pri obhajobe: - **Q:** Prečo len GET? **A:** POST = formuláre, HEAD = cache checks, OPTIONS = API — nereprezentujú navigáciu používateľa. - **Q:** Prečo 2xx–3xx? **A:** 4xx = klient nedostal obsah, 5xx = chyba; len úspešné prístupy. - **Q:** Prečo `Bytes` (group 6) v regexe ale nepoužité? **A:** Dead capture — audit minor finding, ponechané pre kompletnosť.

Úloha 02: robot_ips.py

Teoretický základ: Tretia fáza CRISP-DM je *čistenie súboru od robotov vyhľadávacích služieb*. Prednáška 6 ju explicitne uvádza ako samostatnú časť hrubých dát: `Hrubé dáta → Bez robotov → Rozlíšené sedenia → Doplnené záznamy`. Identifikácia robota cez prístup k `/robots.txt` je heuristika založená na Robots Exclusion Protocol — bežný používateľ tento súbor nikdy nepýta. *(viď prednáška 6 — Čistenie súboru od robotov vyhľadávacích služieb)*

Čo robí: Identifikuje IP adresy, ktoré aspoň raz volali `/robots.txt` (Robots Exclusion Protocol heuristika).

Vstup → výstup: `in/wm2020projekt_studeny.log` (~910k) → `out/02_robot_ips.csv` (~2 354 riadkov, 250 unikátnych IP).

Kľúčové kroky: 1. Rýchly `if "robots.txt" not in line.lower(): continue` (väčšina riadkov skip) 2. Až potom drahý `pattern.match(line)` 3. Double-check: `"robots.txt" in url.lower()` (nie v Referrer/Agent) 4. Append `{"IP": ...}` (duplikáty ponechané — downstream `.unique()`)

Dôležité parametre: - `line.lower()` pred substring testom — case-insensitive (`/Robots.txt`) - Duplikáty NIE drop — každý riadok = 1 prístup, drop až vo filter step

Vzorec / formula: žiadny — len heuristika.

Časté otázky pri obhajobe: - **Q:** Prečo robots.txt = robot? **A:** RFC 9309 — len boti čítajú robots.txt; bežný používateľ/prehliadač ho nikdy nepýta. - **Q:** Koľko unikátnych IP? **A:** 250 (nunique), 2 354 záznamov vrátane duplikátov.

Úloha 03: filter_robotov.py

Teoretický základ: Pokračovanie fázy *čistenie od robotov* — Prednáška 6 hovorí o odstránení vyhľadávacích služieb pred identifikáciou používateľov. Kombinujeme 4 nezávislé heuristiky: signatúry v User-Agent (`bot|crawl|spider`), automatizované nástroje (`curl|wget|libwww`), prázdny UA a IP blacklist z úlohy 02. Cieľom je minimalizovať false negatives (nepustiť bota ďalej) za cenu možných false positives. *(viď prednáška 6 — Čistenie súboru od robotov; prednáška 8 — Identifikácia používateľov na základe User-Agent)*

Čo robí: 4-vrstvový filter botov: UA-bot, UA-auto-tools, prázdny UA, IP-blacklist z úlohy 02.

Vstup → výstup: `01_cleaned_log.csv` (211k) + `02_robot_ips.csv` → `out/03_cleaned_log.csv` (~73 927, -65 %).

Kľúčové kroky: 1. `str.contains(r"bot|crawl|spider", case=False, na=False)` → bot 2. `str.contains(r"cron|curl|wget|libwww|ltx71", ...)` → auto tools 3. `df["Agent"] != "-"` → odstráň prázdne UA 4. `df["IP"].isin(robot_ips)` + `~` → IP-blacklist filter 5. `reset_index(drop=True)` + zapis

Dôležité parametre: - `na=False` — NaN → False (inak by `df[~NaN_mask]` hodil chybu) - `case=False` — `BOT`, `Bot`, `BoT` všetko match - `~` (bitwise NOT) — nie Python `not` (skalárny, hodí ValueError nad Series) - `.isin(ndarray)` — hash-set lookup O(1) na riadok

Vzorec / formula: boolean masky a inverzia.

Časté otázky pri obhajobe: - **Q:** Prečo regex `bot|crawl|spider`? **A:** Univerzálne substringy v UA: Googlebot, AhrefsBot, Baiduspider, Yahoo Slurp... - **Q:** False positive? **A:** „Cubot“ (telefón) — preferujeme recall, radšej obeť ako bot v dátach. - **Q:** Koľko ostane? **A:** ~73 927 (-65 % od 211k).

Úloha 04: obdobie_a_unixtime.py

Teoretický základ: Predpríprava pre identifikáciu sedení na základe času — potrebujeme jednotnú číselnú reprezentáciu času, aby sme mohli počítať rozdiely medzi po sebe nasledujúcimi requestmi. Prednáška 6 a 8 pri definícii sedenia explicitne pracujú s pojmom *časová*

známka (`t_i`) a rozdielmi (`t_i - t_(i-1) ≤ δ`). Bez UNIXTIME by sme nevedeli urobiť ani Length, ani STT/STT_Q test. *(viď prednáška 6 a 8 — Identifikácia sedení na základe času)*

Čo robí: Filter na 7 pridelených dní + vytvorenie premennej UNIXTIME (epoch seconds).

Vstup → výstup: `03_cleaned_log.csv` → `out/04_obdobie.csv` (~73 927 riadkov, +1 stĺpec UNIXTIME).

Kľúčové kroky: 1. 7× `df["DateTime"].str.startswith("DD/Dec/2017")` spojené `|` → OR-maska 2. `df[mask].reset_index(drop=True)` 3. Funkcia `to_unixtime(dt_str)`: split `:` → `[date, hour, min, sec+TZ]`, split `/` → `[DD, Mon, YYYY]` 4. Lookup `months={"Jan":1,...,"Dec":12}` (O(1) hash) 5. `datetime(year, month, day, h, m, s).timestamp()` → float epoch 6. `df["UNIXTIME"] = df["DateTime"].apply(to_unixtime)`

Dôležité parametre: - `str.startswith()` — C-rýchly, presný prefix bez regex overhead - `|` (bitwise OR) na Series, nie `or` - `low_memory=False` — konzistentná dtype inference

Vzorec / formula:

```
UNIXTIME = epoch seconds od 1970-01-01 00:00:00 UTC
December 2017 → ~1.51e9
```

Časté otázky pri obhajobe: - **Q:** Prečo `naive datetime` (bez tzinfo)? **A:** Audit MAJOR — TZ offset `+0100` sa zahadzuje pri split, `.timestamp()` použije lokálnu TZ systému. Pre rozdiely v rámci jedného datasetu OK. - **Q:** Prečo `apply()` namiesto `pd.to_datetime`? **A:** Anti-drift — `to_datetime` rieši TZ inak.

Úloha 05: priradenie_user_id.py

Teoretický základ: Štvrtá fáza CRISP-DM je *identifikácia používateľov*. Prednáška 8 uvádza spôsoby: *„na základe rôznej verzie internetového prehliadača, na základe HTTP/web/Internet/browser cookie"*. Keďže v Apache logu nemáme cookie, používame Cooley/Mobasher/Srivastava heuristiku: rovnaká dvojica (IP, User-Agent) = ten istý používateľ. Prednáška 8 priamo ukazuje príklad tabuľky `IP | Cookie | Agent` s tými istými hodnotami pre všetky riadky → jeden používateľ. *(viď prednáška 8 — Identifikácia používateľov na základe rôznej verzie prehliadača)*

Čo robí: Pridelí USER_ID každému záznamu na základe dvojice (IP, Agent) — Cooley/Mobasher/Srivastava heuristika.

Vstup → výstup: `04_obdobie.csv` (73 927) → `out/05_users.csv` (+ stĺpec USER_ID, 14 563 unikátnych používateľov).

Kľúčové kroky: 1. `sort_values(by=["IP", "Agent", "UNIXTIME"])` — chronologicky v rámci používateľa 2. `reset_index(drop=True)` — kontinuálny RangeIndex 3. `df["USER_ID"] = df.groupby(["IP", "Agent"]).ngroup() + 1`

Dôležité parametre: - `ngroup()` — integer ID skupiny (0..N-1), `+1` → 1..N (kozmetika) - Trojnásobný sort = chronologické rozdelenie v rámci používateľa (nutné pre `shift(1)` v 06)

Vzorec / formula:

```
USER_ID = ngroup(IP, Agent) + 1
```

Časté otázky pri obhajobe: - **Q:** Nedokonalosti? **A:** NAT/proxy → viacero ľudí = 1 user (false negative); mobilný používateľ mení IP → 1 človek = N userov (false positive); UA spoofing. Prednáška 3 priamo ukazuje schémy *„Priame pripojenie do internetu / Pripojenie prostredníctvom proxy servera / Vyrovnávača záťaže"* ako kontext týchto problémov *(viď prednáška 3)*. - **Q:** Prečo sort PRED ngroup? **A:** Aby v ďalšej úlohe `shift(1)` vrátil naozaj predchádzajúci časový záznam. - **Q:** Koľko používateľov? **A:** 14 563 z 73 927 záznamov.

Úloha 06: vypocet_length.py

Teoretický základ: Length (dĺžka návštevy stránky) je centrálna premenná pre obe metódy identifikácie sedení. Prednáška 8 ju definuje pri STT (Session Timeout Threshold): *„Ak po dvoch po sebe nasledujúcich záznamoch z S nerovnosť $t_i - t_{(i-1)} \leq \delta$ neplatí, tak záznamy patria do dvoch rozdielnych sedení."* Zadanie nastavuje δ = STT = 60 min = 3600 s. Length zároveň slúži ako vstup pre KS test exponenciálneho rozdelenia (prednáška 7, 8). *(viď prednáška 8 — Identifikácia sedení na základe času; STT)*

Čo robí: Vypočíta Length = čas na stránke ako rozdiel UNIXTIME medzi sebou nasledujúcimi requestmi toho istého používateľa. STT pravidlo: ≥ 3600 s → NaN.

Vstup → výstup: `05_users.csv` → `out/06_length.csv` (+ stĺpec Length, rovnakých 73 927 riadkov).

Kľúčové kroky: 1. `df["PREV_UNIXTIME"] = df.groupby("USER_ID")["UNIXTIME"].shift(1)` 2. `df["Length"] = df["UNIXTIME"] - df["PREV_UNIXTIME"]` (NaN propagácia) 3. `df.loc[df["Length"] >= 3600, "Length"] = None` (STT 60 min) 4. `drop(columns=["PREV_UNIXTIME"])`

Dôležité parametre: - `groupby(...).shift(1)` — posun JE OBMEDZENÝ na skupinu, prvý riadok = NaN - `>= 3600` — STT (Session Timeout Threshold) zo zadania (60 min) - NaN − čokoľvek = NaN (automatická propagácia)

Vzorec / formula:

```
Length_i = UNIXTIME_i - UNIXTIME_(i-1)   (per USER_ID)
ak Length_i >= 3600s → NaN  (STT pravidlo: 60 min = koniec sedenia)
```

Časté otázky pri obhajobe: - **Q:** Prečo prvý záznam = NaN? **A:** Nemá predchodcu (shift dáva NaN), Length neviem. - **Q:** Prečo 60 min a nie 30 min? **A:** Zadanie explicitne hovorí 60 min STT. Prednáška 8 hovorí všeobecne o *odhade veľkosti časového okna δ* — konkrétnu hodnotu určuje úloha. - **Q:** Prečo `None` v numerickom stĺpci? **A:** Pandas konvertuje na NaN; audit minor (idiomaticky `np.nan`).

Úloha 07: sedenia_rlength.py

Teoretický základ: Toto je jadrová úloha celej témy a najčastejšie otázka pri obhajobe. Prednáška 7 a 8 odvodzujú: *„Premenná X má exponenciálne rozdelenie f(x) = λe^(−λx), F(x) = 1 − e^(−λx). Ak γ je relatívna početnosť navigačných stránok, na odhad hraničného času τ môžeme použiť kvantilovú funkciu $F^{-1}(\gamma) = \tau = -\ln(1-\gamma)/\lambda$. Maximálne vierohodný odhad parametra λ je λ = 1/x̄."* Po dosadení λ = 1/mean dostávame priamo `RLength = −ln(1−p) · mean`. Ak X NEMÁ exponenciálne rozdelenie, prednáška 8 dáva fallback: *„$\tau = Q = Q_3 + 1{,}5 \cdot IQR$"*. Kolmogorov-Smirnov test rozhoduje, ktorú formulu použiť. *(viď prednáška 7 a 8 — Identifikácia sedení na základe času; kvantilová funkcia)*

Čo robí: Identifikácia sedení metódou Reference Length s STT_Q fallbackom (KS test rozhodne, ktorá).

Vstup → výstup: `06_length.csv` → `out/07_sedenia_rlength.csv` (+ SESSION_ID) + `out/07_test_zaver.txt`.

Kľúčové kroky: 1. `length_data = df["Length"].dropna()` (NaN nepatrí do testu) 2. `mean_length = length_data.mean()` (= 1/lambda pre exp.) 3. `kstest(length_data, "expon", args=(0, mean_length))` → (D, p) 4. **Ak p ≥ 0.05** → RLength: `prah = -np.log(1-0.4) * mean_length` 5. **Ak p < 0.05** → STT_Q: `prah = Q3 + 1.5*IQR` 6. `df["NEW_SESSION"] = df["Length"].isna() | (df["Length"] >= prah)` 7. `df["SESSION_ID"] = df["NEW_SESSION"].cumsum()`

Dôležité parametre: - `np.log` = PRIRODZENÝ logaritmus (ln), NIE log10! - `kstest(..., "expon", args=(loc=0, scale=mean))` — scale = 1/lambda - `p = 0.4` (40 % navigačných stránok podľa zadania) - `cumsum()` na bool = vektorizovaný „session counter"

Vzorec / formula:

```
RLength = -ln(1 - p) * mean(Length)
        = -ln(0.6) * mean ≈ 0.5108 * mean   (pre p=0.4)
STT_Q   = Q3 + 1.5 * IQR
```

```
KS:     H0: data ~ Exp(scale=mean); D = sup|F_emp - F_teor|
        p < 0.05 → zamietame H0
```

Časté otázky pri obhajobe: - **Q:** Prečo exponenciálne rozdelenie? **A:** Predpoklad: navigačné Length sú krátke pauzy = nezávislé udalosti (Poissonov proces). Prednáška 7 a 8 explicitne pracuje s f(x) = λe^(−λx) ako modelom dĺžky návštev *(viď prednáška 7 a 8)*. - **Q:** Odvoď RLength zo CDF. **A:** F(x) = 1 − e^(−λx), invertujem: γ = 1 − e^(−λτ) ⇒ e^(−λτ) = 1 − γ ⇒ −λτ = ln(1−γ) ⇒ τ = −ln(1−γ)/λ. Dosadím λ = 1/x̄ ⇒ τ = −ln(1−γ)·x̄. To je presne `RLength`. - **Q:** Prečo 1.5·IQR? **A:** Tukeyho horná ohrada — mierne outliery v boxplote. Prednáška 8 to uvádza ako alternatívny odhad keď exponenciálne rozdelenie neplatí. - **Q:** Prečo cumsum nad bool? **A:** False=0, True=1; každé True inkrementuje SESSION_ID → riadky medzi 2 True majú rovnaké ID.

Úloha 08: sedenia_href.py

Teoretický základ: Metóda h-ref (heuristic referrer) je *identifikácia sedení na základe štruktúry prehľadávaných stránok* — prednáška 6, 7 a 8 ju uvádza ako tretí spôsob popri času. Klasická Cooley h-ref definícia: dva po sebe idúce požiadavky patria do toho istého sedenia, ak Referrer aktuálneho requestu je medzi predtým navštívenými URL v sedení. V našom zjednodušení (audit MAJOR) testujeme len `Referrer == "-"` ako proxy pre „priamy vstup". *(viď prednáška 6 a 8 — Identifikácia sedení na základe štruktúry; ELF / Extended Log File)*

Čo robí: Identifikácia sedení metódou h-ref (heuristic referrer): NaN Length OR Length≥mean OR Referrer="-".

Vstup → výstup: `06_length.csv` → `out/08_sedenia_href.csv` (+ SESSION_ID).

Kľúčové kroky: 1. `mean_length = df["Length"].mean()` (globálne časové okno) 2. 3 OR podmienky: - `df["Length"].isna()` (hranica medzi používateľmi) - `df["Length"] >= mean_length` (prekročený priemer) - `df["Referrer"] == "-"` (priamy vstup, bookmark) 3. `df["SESSION_ID"] = NEW_SESSION.cumsum()`

Dôležité parametre: - `|` (NIE `or`) — element-wise OR nad Series - Zátvorky okolo každej podmienky NUTNÉ (`|` má vyššiu prioritu ako `==`, `>=`) - `Referrer == "-"` — Apache marker pre „no referrer"

Vzorec / formula:

```
NEW_SESSION = isna(Length) OR (Length >= mean) OR (Referrer == "-")
SESSION_ID  = cumsum(NEW_SESSION)
```

Časté otázky pri obhajobe: - **Q:** V čom sa líši od klasického Cooley h-ref? **A:** Audit MAJOR — klasika porovnáva Referrer s predtým navštívenými URL v sedení (prednáška 8: *„Ak sú odkazujúce stránky dostupné (ELF), tak sedenie je sekvencia… p_i = r_(i+1)"*); tu len `=="-"`. Externý referrer z

inej domény nevyvolá break. - **Q:** Prečo OR a nie AND? **A:** Stačí jeden signál — nový používateľ, dlhá pauza ALEBO priamy vstup = nové sedenie.

Úloha 09: doplnenie_rlength.py

Teoretický základ: Posledná fáza CRISP-DM je *dopĺňanie ciest* (path completion). Prednáška 8 ukazuje schému: máme aktuálnu navigáciu (stránka A → B → C → D → X) a mapu webu (dvojice URL → odkazujúca stránka). Ak v sedení medzi dvoma stránkami chýba prepojenie (browser cache, back button), použijeme mapu webu na rekonštrukciu medzistránok. Príklad z prednášky 8: zo záznamov `A → B → C → D → X` sa pomocou mapy obnoví celá cesta `A B C D C B A X`. *(viď prednáška 8 — Dopĺňanie ciest, schéma mapy webu)*

Čo robí: Path completion pre RLength sedenia — doplní chýbajúce medzistránky cez mapu webu postavenú z logu.

Vstup → výstup: `07_sedenia_rlength.csv` → `out/09_doplnene_rlength.csv` (~73 927 + ~4 031 doplnených).

Kľúčové kroky: 1. `url_to_path()` — normalizuje plnú URL na cestu (urlparse → path) 2. Build mapa webu: dvojice (Referrer, URL) z logu, filter `"ukf.sk"`, drop NaN 3. `groupby("URL")["ref_path"].agg(lambda x: x.value_counts().idxmax())` → najčastejší parent 4. `parent = dict`, `map_paths = set(parent.keys())` (O(1) lookup) 5. `to_dict("records")` → ~10–20× rýchlejšia iterácia ako iterrows 6. Pre dvojicu (curr, next): rovnaké SESSION_ID + url_from != url_to + `url_from NOT IN map AND url_to IN map` → vlož `parent[url_to]` ako medzistránku

Dôležité parametre: - `urlparse(url).path` — odreže scheme/netloc/query → kanonická cesta - `case=False, na=False` v `str.contains("ukf.sk", ...)` - `dict.get()` defensive lookup

Vzorec / formula: žiadny — heuristika.

Časté otázky pri obhajobe: - **Q:** Prečo mapa z logu a nie z Excelu? **A:** V projekte chýba externý zdroj — staviame z dvojíc (Referrer, URL) v samotnom logu. Prednáška 2 uvádza ako zdroje dát aj *„Web → Obsah, Mapa webu, Logovacie súbory"* — my máme len logy, takže mapu odvodzujeme z nich. - **Q:** Prečo `to_dict("records")`? **A:** `iterrows()` vyrobí Series objekt v každom kroku — dict iterácia je 10–20× rýchlejšia. - **Q:** Audit findings? **A:** Logika je INVERZNÁ voči klasike (membership v mape vs. porovnanie hrany); doplní len 1 medzistránku (nie rekurzívne).

Úloha 10: doplnenie_href.py

Teoretický základ: Identická fáza *dopĺňanie ciest* ako úloha 09, len aplikovaná na sedenia identifikované h-ref metódou. Cieľom je porovnateľnosť oboch metód identifikácie sedení na úrovni doplnených ciest — výsledok závisí od štruktúry SESSION_ID. Prednáška 6, 7 a 8 dôsledne

odlišuje *identifikáciu sedení na základe času* (RLength, STT) a *na základe štruktúry* (h-ref) ako dve nezávislé vetvy fáz procesu. *(viď prednáška 6, 7, 8 — fázy procesu Web Log Miningu)*

Čo robí: Path completion pre h-ref sedenia — rovnaký algoritmus ako úloha 09, len iný vstup.

Vstup → výstup: `08_sedenia_href.csv` → `out/10_doplnene_href.csv` (~73 927 + ~0.19 % doplnených).

Kľúčové kroky: identické s úlohou 09, len iný `SESSION_ID` zdroj (h-ref namiesto RLength).

Dôležité parametre: identické — `urlparse`, `groupby+agg(value_counts().idxmax())`, `to_dict("records")`.

Vzorec / formula: žiadny.

Časté otázky pri obhajobe: - **Q:** Prečo dva samostatné skripty (09 + 10)? **A:** Zadanie chce path completion pre OBE metódy oddelene; výsledok závisí od SESSION_ID z 07 vs. 08. - **Q:** Prečo menej doplnení pri h-ref? **A:** h-ref produkuje kratšie sedenia (3 podmienky) → menej dvojíc v rámci 1 sedenia → menej príležitostí na doplnenie. - **Q:** Čo zdedí doplnená stránka? **A:** Všetky polia z `curr` (UNIXTIME, USER_ID, SESSION_ID, IP, Agent) — mení sa LEN URL. Length sa neprepočítava (audit major).

🎯 Vyhodnotenie / na čo si dať pozor pri obhajobe

1. **Vzorec RLength:** `-ln(1-p)*mean` — Munk sa pýta odvodenie z CDF exponenciálneho rozdelenia (prednáška 7, 8: `F(x)=1-e^(-λx)`, invertovať pri γ, dosadiť `λ=1/x̄`).
2. **STT_Q = Q3+1.5·IQR** — keď KS p < 0.05; presné odvodenie z Tukeyho boxplotu (prednáška 8: kvartilový odhad keď X nemá exponenciálne rozdelenie).
3. **USER_ID = (IP, Agent)** — nedokonalosti (NAT, mobile, UA spoofing) treba vedieť (prednáška 3, 8).
4. **NaN propagation v Length** — prvý záznam používateľa = NaN, použije sa ako session marker.
5. **cumsum nad bool** — najtypickejší pandas idiom pre SESSION_ID.
6. **Path completion** — prečo (browser cache, back button) a ako (mapa z dvojíc Referrer→URL); pozri schému v prednáške 8.
7. **Audit findings** — vedieť priznať: h-ref simplifikácia (`==-` namiesto členstva v navštívených URL), inverzná path completion logika, naive datetime bez TZ.
8. **CRISP-DM mapping** — vedieť, do ktorej fázy patrí ktorá úloha: získanie (00) → čistenie od nepotrebných dát (01) → čistenie od robotov (02, 03) → identifikácia používateľov (04, 05) → identifikácia sedení (06, 07, 08) → dopĺňanie ciest (09, 10).